

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación

Contenido

Introducción	04
Modelado de casos de uso	05
¿Qué es un caso de uso?	05
Interacción entre actores y casos de uso.....	07
Especificaciones de Flujos en el modelo de Casos de Uso.....	07
Un caso de uso se compone de:	08
Postcondiciones y Post Condiciones	09
Diagrama de casos de uso	10
Relaciones include y extend entre casos de uso	12
Sistemas y Subsistemas en el Análisis de Sistemas	15
Análisis Orientado a Objetos	17
Análisis Estructurado vs. Análisis Orientado a Objetos.....	18
Trazabilidad y Balanceo entre Modelos.....	19
Normalización de Archivos, Diccionarios y Tkinter.....	21
Diccionarios en Python	22
Tkinter básico	24
Tkinter - Widgets.....	26
Bibliografía.....	31

Clase 6



iTe damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- Aprendimos a modelar funcionalmente un sistema a través de casos de uso.
- Vimos cómo utilizar relaciones include y extend.
- Comprendimos la diferencia entre flujos alternativos y extensiones.
- Introdujimos la noción de subsistemas como unidades dentro de un sistema mayor.
- Comparamos dos grandes enfoques: análisis estructurado y orientado a objetos.
- Reforzamos la importancia de la trazabilidad y el balanceo para garantizar coherencia en los modelos.
- Aprendimos cómo se define el punto de entrada de una aplicación con `main()` y por qué conviene usarlo.

- Trabajamos con diccionarios, una estructura muy flexible y fundamental en Python.
- Vimos cómo reutilizar código importando clases desde otros archivos.
- Exploramos los conceptos básicos de Tkinter, abriendo la puerta a crear interfaces gráficas en Python.

Introducción

En la clase anterior trabajamos con la representación estructural de los datos mediante el **Diagrama Entidad-Relación (DER)** y analizamos el proceso de balanceo de modelos, comparando cómo las distintas herramientas (**DFD, DC, DD y DER**) deben mantenerse consistentes entre sí para reflejar correctamente los requerimientos del sistema.

Avanzando en el **modelado funcional**, hoy nos adentramos en un enfoque centrado en el comportamiento del sistema y su interacción con el entorno, abordando los casos de uso, uno de los pilares del lenguaje **UML**. También compararemos los enfoques del **análisis estructurado** y el **análisis orientado a objetos**, sentando las bases para modelar sistemas modernos de manera más intuitiva y robusta.

Modelado de casos de uso

¿Qué es un caso de uso?

Un caso de uso es una descripción detallada de cómo un usuario (o “**actor**”) interactúa con un sistema para lograr un objetivo específico. Se trata de una herramienta central en el modelado funcional que permite capturar requisitos del sistema desde el punto de vista del usuario.

No se enfoca en cómo se realiza internamente la acción, sino en qué espera obtener el usuario del sistema.

Características principales

- Están expresados desde la perspectiva del usuario, no del sistema.
- Se documentan de forma textual, muchas veces en lenguaje natural.
- Enfatizan la interacción entre el usuario y el sistema.
- Cada caso de uso es iniciado por un actor externo.
- El nombre del caso de uso se escribe en infinitivo, indicando la acción que se desea realizar (Se sigue una estructura VERBO + SUSTANTIVO).

Utilidad de los casos de uso:

- Ayudan a documentar las funciones del sistema y el rol de cada actor.
- Favorecen la comunicación entre clientes, usuarios y desarrolladores.
- Sirven como base para definir pruebas funcionales.
- Delimitan el alcance del sistema, permitiendo enfocarse en los escenarios más relevantes.
- Facilitan la elaboración de diagramas de casos de uso en UML.
- Permite generar la documentación de usuario y las pruebas funcionales del sistema, en paralelo con el desarrollo.

Actores

Un actor representa a una persona, sistema externo o dispositivo que interactúa con el sistema que estamos diseñando. Los actores:

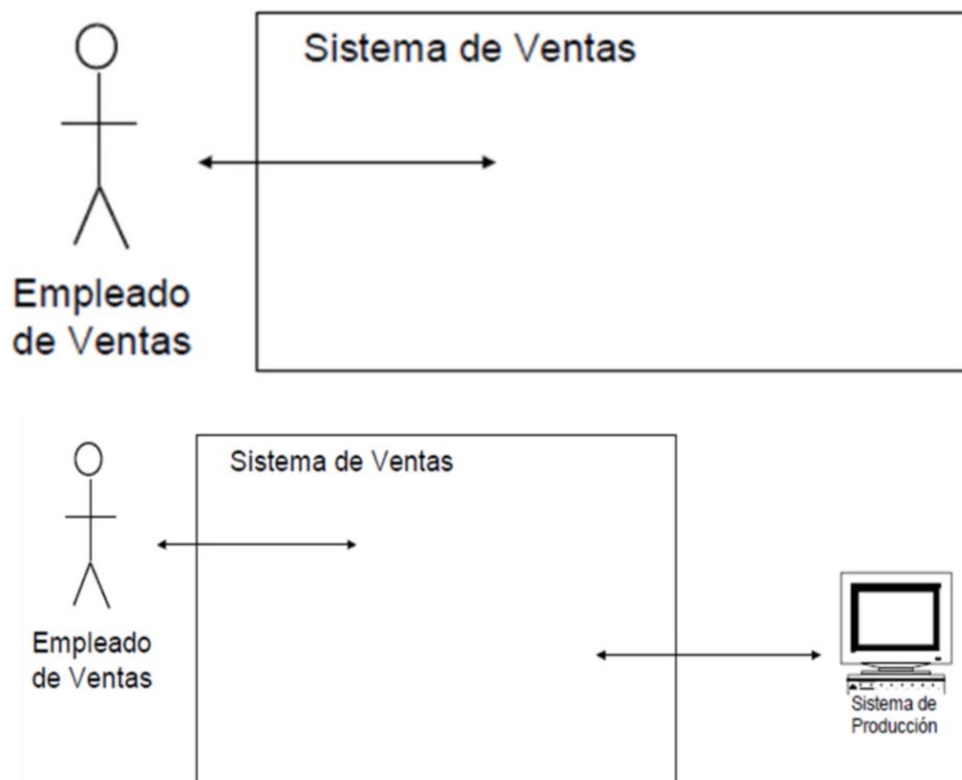
- Son externos al sistema.
- Pueden iniciar casos de uso.
- Se definen por el rol que cumplen, no por el individuo específico.

Ejemplo:

En un sistema de ventas:

Un **operador** que registra pedidos telefónicos es un actor llamado **Empleado de Ventas**.

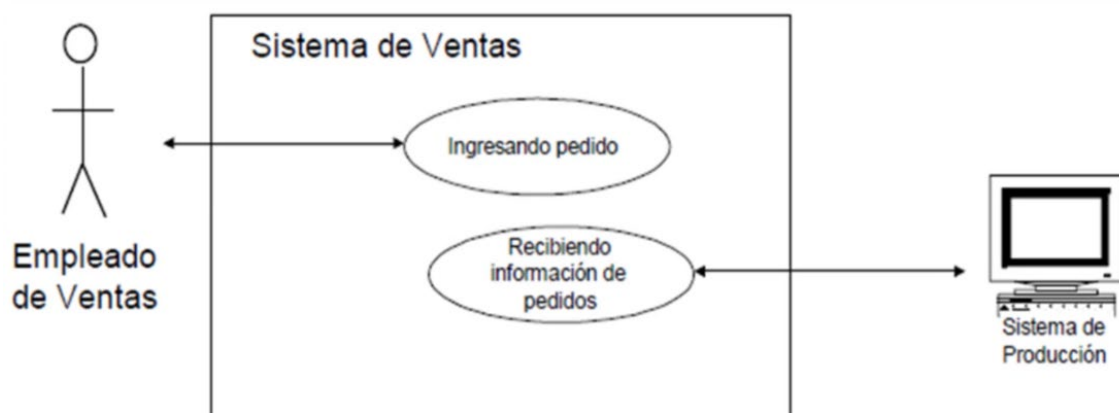
Si varios empleados hacen exactamente lo mismo en el sistema, todos pertenecen al mismo **actor**.



Interacción entre actores y casos de uso

Cada caso de uso se activa por un actor y describe el conjunto de pasos que este realiza junto al sistema para cumplir un objetivo. Es común que también participen otros actores secundarios en la ejecución.

Gráficamente, los casos de uso se representan como óvalos, con su nombre dentro, y los actores como figuras humanas, conectados al caso de uso con líneas.



Especificaciones de Flujos en el modelo de Casos de Uso

Un caso de uso está compuesto por un flujo principal y uno de más flujos alternativos, como se muestran en la siguiente imagen:

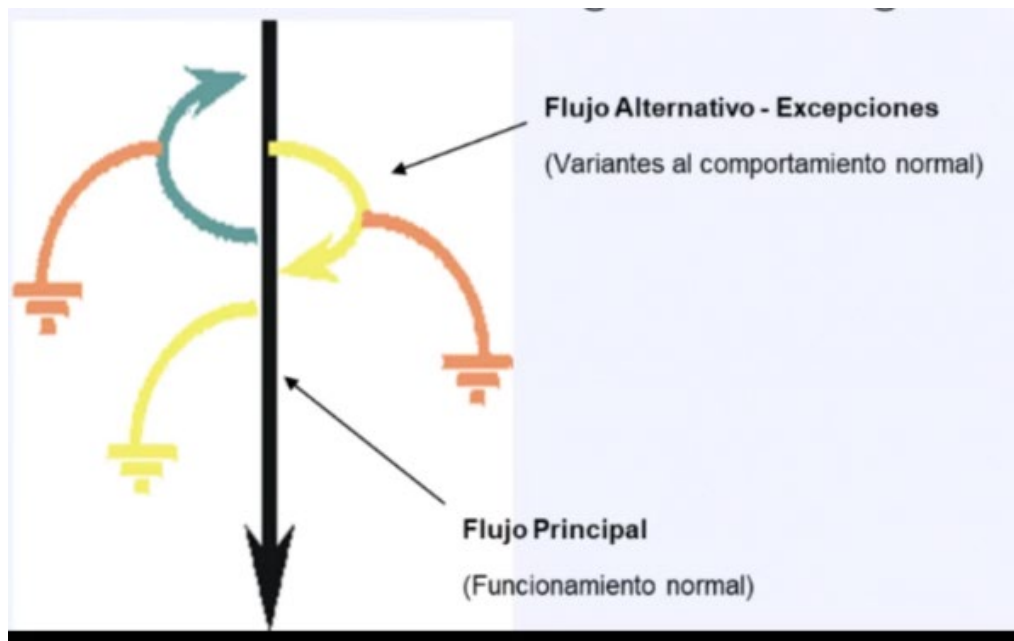


Imagen: Ilustración de Flujos en un Diagrama de Casos de Uso

Un caso de uso se compone de:

Flujo principal (o escenario básico): Describe la secuencia normal de acciones esperadas para completar exitosamente el objetivo.

Ejemplo de un Flujo principal:

Caso de Uso: Procesar Venta

Actor: Cajero

1. El cliente llega con los productos.
2. El cajero inicia un nuevo proceso de venta.
3. Registra los productos escaneando su código.
4. Calcula el total.
5. Solicita el pago.
6. El sistema emite la factura.

Flujo Alternativo

Durante la ejecución de un caso de uso, suelen aparecer pasos opcionales e inclusive errores al curso principal. Esto causa que el flujo principal se vea desviado dando así el nombre de "Flujo Alternativo".

Son variaciones que ocurren cuando se presentan condiciones especiales, errores o decisiones distintas a las del flujo principal.

Ejemplo – Flujos alternativos:

Caso de Uso: Procesar Venta

Flujo Alternativo: Pago en Efectivo

- 5.1. El cajero ingresa el monto recibido.
- 5.2. El sistema calcula el vuelto.
- 5.3. Se entrega el vuelto y se registra el pago.
- 5.4. Vuelve a 6.

Flujo Alternativo: Pago con tarjeta de crédito

- 5.1. El cliente entrega la tarjeta.
- 5.2. El cajero ingresa los datos.
- 5.3. El sistema pide autorización a un servicio externo.
- 5.4. Se valida la tarjeta de crédito.
- 5.5. Se registra el pago realizado.
- 5.6. Vuelve a 6.

Postcondiciones y Post Condiciones

Precondiciones

Indican el estado que debe cumplir el sistema antes de que el caso de uso pueda ejecutarse. No se validan dentro del caso de uso, sino que se asumen como ciertas.

Ejemplo:

Precondición: El cajero debe estar autenticado en el sistema para poder procesar una venta.

Postcondiciones

Describen el estado final del sistema luego de completar el caso de uso con éxito.

Ejemplo:

Postcondición: Se registra la venta, se actualiza el stock, se imprime la factura y se graba el pago.

Diagrama de casos de uso

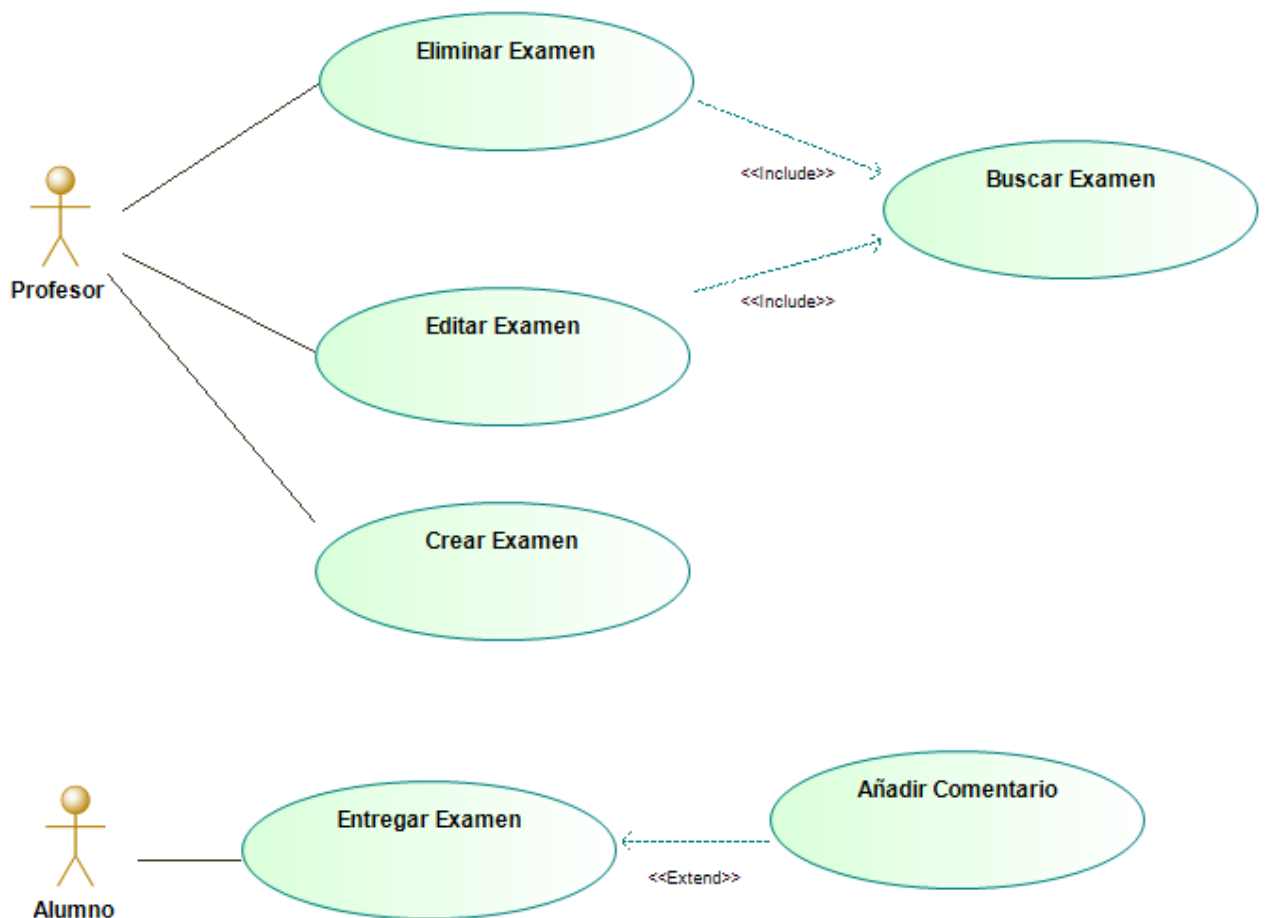


Diagrama de Casos de Uso realizado con Modelio

Especificación de casos de uso

Normalmente se suele utilizar plantillas específicas para realizar las especificaciones de Casos de Uso, donde se incluye no solamente los flujos sino la descripción, pre y post-condiciones, requisitos complementarios, etc.

Caso de Uso	Asignar Habitación.	
Actores	Recepción.	
Descripción	Asignar una habitación al cliente que llega. El cliente puede o no haber efectuado previamente la reserva.	
Precondición		
Flujo Principal		Flujo Alternativo / Excepciones
1	Cliente llega a la Recepción y solicita habitación.	
2	Recepción consulta al cliente si posee reserva.	<u>2 a - Posee reserva</u>
		1. Recepción consulta si está la habitación disponible.
		1.1 Si está disponible, Recepción actualiza el registro de reserva y la asigna.
		1.2 Si No está disponible, Recepción busca habitación en hotel cercano y consigue <u>remis</u> .
		<u>2 b - No posee reserva</u>
		1. Recepción verifica si hay habitaciones disponibles.
		1.1. Si hay habitación la asigna.
		1.2. Si no hay habitación se informa al cliente.
3	Recepción asigna un número de habitación al cliente.	
4	Recepción actualiza el registro de habitaciones ocupadas.	
Observaciones		
Postcondición	Se asigna habitación. Se actualizan registro de reservas y de habitaciones ocupadas.	

Ej. Plantilla 1 de Especificación de Casos de Uso

2.1.1.1. Especificaciones de casos de uso.

2.1.1.1.1. [Número y Nombre del CU que se especificó en el diagrama]

- Historial de versiones del CU:

Versión 	Fecha	Descripción	Autor
[Nro. de la versión del CU]	[Fecha en la cual se libera la versión del CU.]	[Descripción de los cambios realizados en el CU.]	[Nombre y apellido del autor/autores]

- **Descripción:** [Se detalla brevemente que hace el caso de uso.]
- **Actores:** [Se especifican los actores que se encuentran asociados al caso de uso.]
- **Flujo Principal:** [Acá colocan el flujo principal de su caso de uso, es decir, todos los pasos principales.]
- **Flujo alternativo:** [Acá colocan los flujos alternativos existentes, es decir, los pasos alternativos que se ejecutan de su caso de uso.]
- **Excepciones:** [Acá colocan los pasos que se ejecutarán en caso de que haya excepciones que deban especificar.]
- **Requisitos Complementarios:** [Acá colocan los requisitos no funcionales que se complementen con el CU.]

Ej. Plantilla 2 de Especificación de Casos de Uso (Utilizada en materia Ingeniería de Requisitos)

Relaciones include y extend entre casos de uso

Include

La relación include se utiliza cuando varios casos de uso comparten una funcionalidad común (y si el comportamiento a incluir es lo suficientemente complejo como para realizar un caso de uso aparte). En lugar de repetir esa funcionalidad en cada uno, se define un caso de uso separado que se incluye obligatoriamente cada vez que se ejecuta el caso de uso principal.

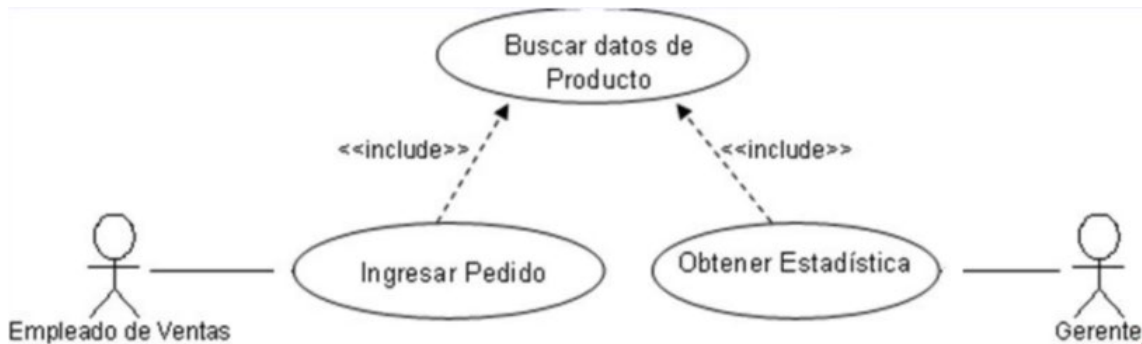
Características:

- Reutiliza pasos comunes.
- El caso incluido siempre se ejecuta.
- Representa comportamiento obligatorio.
- Se detalla en el diagrama su palabra en inglés "Include" (en lugar de "inclusión") y debe estar contenida entre los símbolos << >>.

Ejemplo:

Caso de uso: Ingresar Pedido

Incluye: Buscar datos de Producto



Para expresar una inclusión en la especificación de casos de uso, es importante referenciar al caso de uso que se está incluyendo en el primer paso del caso de uso base.

Extend

La relación "extend" permite añadir funcionalidad adicional bajo ciertas condiciones específicas, siempre y cuando sean opcionales (y representen un comportamiento complejo de varios pasos). La ejecución del caso extendido no es obligatoria; depende de que se cumpla una condición.

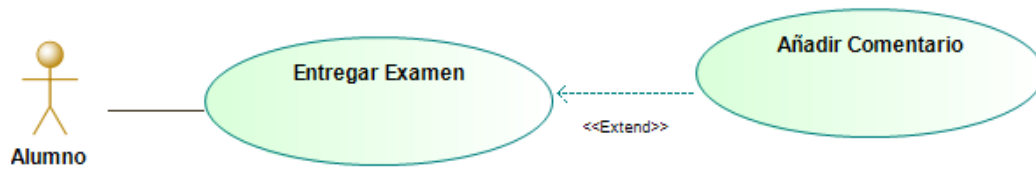
Características:

- Representa funcionalidad opcional.
- Se ejecuta solo si se dan ciertos escenarios.
- Ideal para casos especiales o alternativos.
- Se detalla en el diagrama su palabra en inglés "Extend" (en lugar de "extensión") y debe estar contenida entre los símbolos << >> ,

Ejemplo:

Caso de uso: Entregar Examen

Extiende: Añadir Comentario



La pregunta que nos podemos hacer ahora es: ¿Cuál es la diferencia entre una alternativa y una extensión?

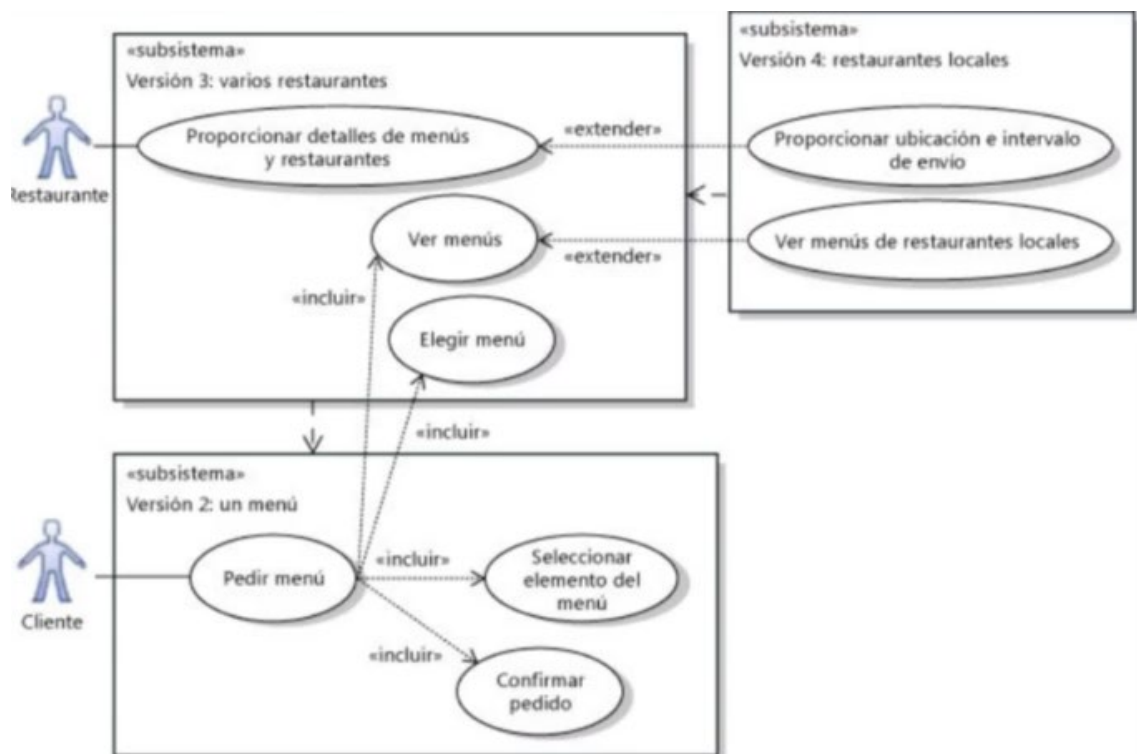
Diferencia entre flujo alternativo y extensión

La respuesta puede derivarse de las características de cada uno:

- Una extensión es un caso de uso en sí mismo, mientras que una alternativa no.
- Una alternativa es un error o excepción, mientras que una extensión puede no serlo.

Como regla aproximada en este caso podemos pensar que si algo opcional debe ser expresado con más de un paso, seguramente es una extensión y no un flujo alternativo.

Dentro de los diagramas de casos de uso podemos expresar sistemas y subsistemas!, pero.. ¿Qué son?



Sistemas y Subsistemas en el Análisis de Sistemas

Sistemas: Por definición, sistema es un conjunto de partes relacionadas que interactúan entre sí para llegar a un objetivo. Por ejemplo: Sistema solar, sistema de software, sistema nervioso, etc.

Subsistemas: Un subsistema es una parte funcional del sistema general, que cumple objetivos propios y se puede analizar de manera independiente.

Ejemplo:

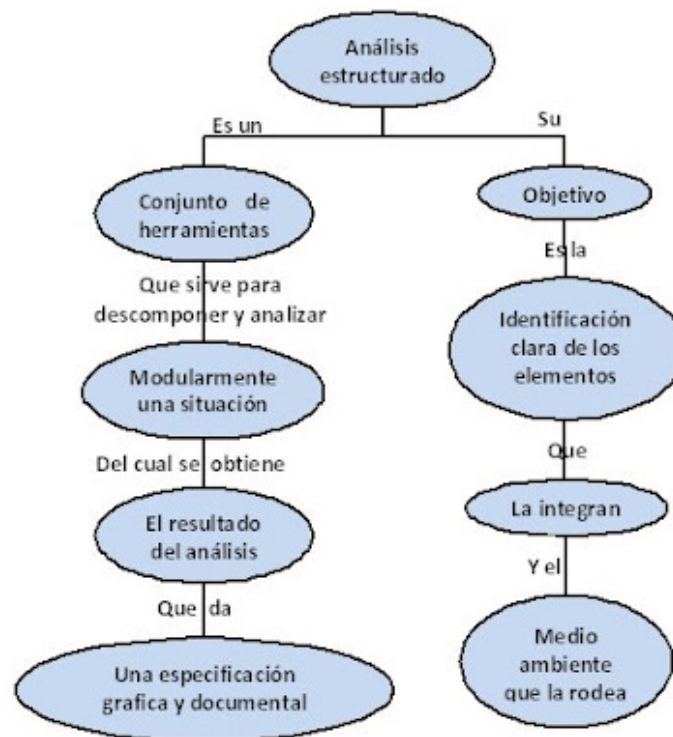
Sistema: Sistema de gestión escolar

Subsistemas:

- Gestión Académica.
- Matrícula de alumnos.
- Administración y Finanzas.

¿Cómo influye esto en el análisis de sistemas?

Es importante tener en cuenta que tenemos dos paradigmas en este ámbito, el análisis estructurado y el análisis orientado a objetos, veamos cada uno de ellos:



El análisis estructurado se enfoca en los procesos y en los flujos de datos que atraviesan un sistema. Es un enfoque que descompone un problema en funciones o procedimientos más pequeños y maneja las interacciones entre ellos. Aquí, el objetivo es entender qué hace el sistema, es decir, describir el comportamiento del sistema y los pasos necesarios para cumplir con los requisitos.

Características principales:

- Se utiliza una serie de herramientas y diagramas, como diagramas de flujo y diagramas de contexto, para describir los procesos, el flujo de información y las interacciones entre diferentes partes del sistema.
- Es más adecuado para sistemas que se pueden describir como una secuencia de actividades o procesos.
- El foco está en la descomposición funcional. Por ejemplo, se tomaría una tarea grande (como "Gestión Académica") y se dividiría en subtareas más pequeñas y manejables.
- Utiliza herramientas como: Diagrama de Contexto (DC), Diagrama de Flujo de Datos (DFD), diccionario de datos (DD) y Diagrama Entidad-Relación (DER).

Importancia del concepto de sistema/subsistema:

- Cada función principal se puede analizar como un subsistema funcional.
- Esto permite una visión jerárquica del sistema.
- Se promueve la modularidad: cada módulo tiene una responsabilidad específica.

Ejemplo:

Un sistema de gestión académica puede dividirse en subsistemas como gestión de alumnos, inscripciones, cursado y evaluación.

Análisis Orientado a Objetos

El análisis orientado a objetos adopta una visión diferente, centrándose en los objetos del sistema que encapsulan tanto los datos como los comportamientos.

Encontrarán mucha similitud con la Programación Orientada a Objetos, puesto que ambos parten de la misma premisa:

En lugar de describir el sistema como un conjunto de procesos, lo describe como una colección de objetos que interactúan entre sí.

Un objeto es una entidad que representa un concepto del mundo real o del dominio del problema (por ejemplo, "Libro" o "Usuario" en un sistema de biblioteca, cada uno con sus respectivos atributos).

Características principales:

- Se basa en objetos que combinan datos y comportamientos.
- Utiliza herramientas como: **Diagrama de Clases, Diagrama de Casos de Uso, Diagramas de Secuencia.**
- Se enfoca en cómo se organiza el sistema internamente.
- Es más apropiado para sistemas complejos y evolutivos.

Análisis Estructurado vs. Análisis Orientado a Objetos

Característica	Análisis Estructurado	Programación Orientada a Objetos
Enfoque	Funcional, procesos y flujos de datos. Centrado en el "qué" hace el sistema.	Basado en objetos que encapsulan datos y comportamientos. Centrado en el "cómo" lo hace el sistema.
Unidad principal de análisis	Funciones existentes y procesos	Clases y objetos
Representación visual	Modelos como Diagramas de flujo de datos (DFD), Diagrama de Contexto (DC), etc.	Diagramas de clases, casos de uso, etc.
Organización del código	Modularidad funcional	Encapsulamiento, herencia, polimorfismo
Ventaja	Claridad de procesos y flujos	Reutilización, abstracción, flexibilidad

Ejemplo de ambos paradigmas:

En un sistema de biblioteca, el análisis estructurado describiría los procesos de préstamo de libros, cómo se componen los datos, etc, mientras que el análisis orientado a objetos definiría clases como "Libro" y "Usuario".

¿Por qué es importante?

La trazabilidad asegura que **todos los requisitos se cumplen** a lo largo del ciclo de vida del proyecto. Permite verificar que los modelos, diagramas, y el sistema final estén alineados con los requisitos originales, evitando que se pierdan o malinterpreten.

Ejemplo de trazabilidad

Si uno de los requisitos del sistema es que "El usuario puede visualizar el historial de pedidos", la trazabilidad garantizará que este requisito:

- Se vea reflejado en los modelos de flujo de datos (DFD, Diagrama de Flujo de Datos), donde se muestra el proceso que permite al usuario acceder a su historial.
- Se representa en el Diagrama Entidad-Relación (ER), con una relación clara entre las entidades "Usuario" e "Historial de Pedidos".
- Se incluye en el diccionario de datos, que especifica los atributos y detalles de las entidades y su relación.

La trazabilidad permite verificar que este requisito específico se documenta y se sigue en cada fase del desarrollo, asegurando que se implemente correctamente.



¡SEGUIMOS CON PYTHON!

Función main, Diccionarios,
Importación de Archivos y

Normalización de Archivos, Diccionarios y Tkinter

Vamos a profundizar en algunos conceptos fundamentales para el desarrollo de aplicaciones en Python. Trabajaremos con:

- La función `main()` como punto de entrada del programa.
- Estructuras de datos clave, como los diccionarios.
- La forma correcta de importar clases o funciones desde otros archivos.
- Una introducción básica a Tkinter, la librería gráfica de Python.

Aunque algunos temas como Tkinter no fueron abordados activamente en la cursada, es útil conocerlos como base para futuros desarrollos más visuales.

main() en Python

A diferencia de otros lenguajes como C, C++ o Java, Python no requiere una función principal obligatoria. Sin embargo, es una buena práctica definir una función `main()` para estructurar mejor el código, sobre todo en programas más grandes.

Se suele usar junto con una verificación para asegurarse de que el código se ejecute solo si el archivo es ejecutado directamente, no cuando se importa como módulo en otro programa. Esto se hace usando la estructura:

¿Qué significa `if __name__ == "__main__":`?

En Python, cada archivo `.py` tiene una variable especial llamada `__name__`. Cuando un archivo se ejecuta directamente, esa variable toma el valor `"__main__"`. Si el archivo se importa desde otro módulo, el valor será el nombre del módulo.

Main aplicación

```
1  def main():
2      print("Hola, este es el programa principal.")
3
4  if __name__ == "__main__":
5      main()
```

- **__name__**: Es una variable especial en Python que toma el valor "__main__" si el script se está ejecutando directamente.
- **if __name__ == "__main__":**: Comprueba si el archivo actual está siendo ejecutado como programa principal. Si es así, ejecuta la función main(). Si el archivo es importado en otro módulo, el bloque dentro del if no se ejecutará.

¿Por qué es útil?

- Permite que el archivo se **ejecute directamente** como programa.
- A la vez, permite que sus funciones y clases sean **reutilizables** desde otros archivos sin que se ejecute automáticamente.

Diccionarios en Python

Un **diccionario** en Python es una estructura de datos que almacena pares clave-valor. Su funcionamiento es muy parecido al de los objetos JSON en JavaScript.



Características:

- La **clave** (key) es única e inmutable (puede ser un string, número, o tupla).
- El **valor** (value) puede ser cualquier tipo de dato, incluso listas o diccionarios anidados.

- Se define entre **llaves {}**, y dentro de esas llaves, cada elemento se escribe como un par clave: valor, separados por comas.

Ejemplo de Código:

```
precios = {  
    "Manzana": 1.50,  
    "Banana": 0.75,  
    "Naranja": 1.00  
}  
  
# Acceder a un valor:  
print(precios["Manzana"]) # Salida: 1.50
```

Dentro del propio diccionario podemos agregar nuevos elementos, modificar los que existen o eliminar.

```
# Agregar o modificar  
precios["Pera"] = 1.20  
  
# Eliminar  
del precios["Banana"]  
  
# Recorrer diccionario  
for fruta, precio in precios.items():  
    print(f"{fruta}: ${precio}")
```

Import de archivos

En Python, es muy común dividir el código en módulos, es decir, distintos archivos que contienen funciones o clases reutilizables. Para usar código desde otro archivo, se utiliza la palabra clave **import**.

Ejemplo:

Supongamos que tenés una clase **Autenticacion** definida en un archivo llamado **autenticacion.py**.

```
1  # autenticacion.py
2  class Autenticacion:
3      def login(self, usuario, contraseña):
4          print(f"Iniciando sesión de {usuario}")
```

Podemos importar esta clase desde otro archivo de la siguiente manera:

```
1  # main.py
2  from autenticacion import Autenticacion
3
4  def main():
5      auth = Autenticacion()
6      auth.login("admin", "1234")
7
8  if __name__ == "__main__":
9      main()
10
```

#Salida: "Iniciando sesión de admin"

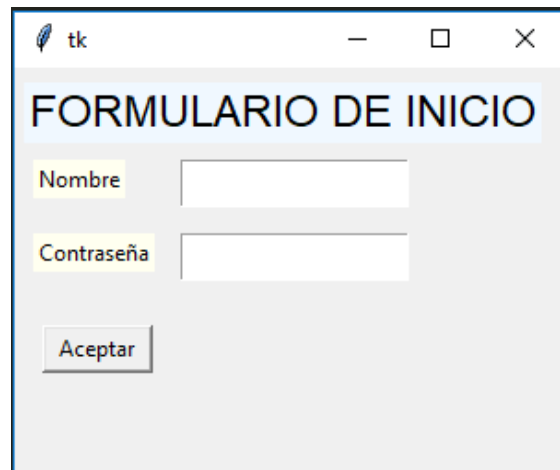
Tkinter básico

Como breve introducción, Tkinter es la librería más equivalente a Java Swing que podemos encontrar en Python. Nuestro objetivo no es hacer un programa de consola, por lo que no vimos desde un inicio este contenido de manera activa. Sin embargo, para que lo conozcan y sepan de qué se trata, vamos a ver varios aspectos interesantes:



Tkinter - Ventanas

Ventana principal (Tk): La ventana principal de una aplicación en Tkinter es creada por el objeto Tk(). Esta ventana es donde aparecerán todos los widgets (elementos de la interfaz gráfica como botones, etiquetas, etc.). Es lo primero que debes crear al desarrollar una GUI con Tkinter. (Como el frame de **jframe**)



Ejemplo de ventana creada con Tkinter

```
1  #Paso importamos el modulo
2  import tkinter as tk
3
4  def main():
5      ventana = tk.Tk()
6      ventana.title("Mi Ventana Tkinter")
7      ventana.geometry("300x200")
8      ventana.mainloop()
9
10 if __name__ == "__main__":
11     main()
```

Explicación del código:

- **import tkinter as tk:** Importa la biblioteca Tkinter y la renombra como tk para facilitar su uso.
- **ventana = tk.Tk():** Crea un objeto Tk que representa la ventana principal.
- **ventana.title("Mi Ventana"):** Establece el título de la ventana.
- **ventana.geometry("300x200"):** Define el tamaño de la ventana en píxeles (ancho x alto).

- **ventana.mainloop():** Inicia el bucle principal de eventos de Tkinter. Este bucle es esencial para que la ventana se muestre y responda a las acciones del usuario (como clics, pulsaciones de teclas, etc.).

Tkinter - Widgets

Widgets: Los widgets son los componentes visuales de la GUI. Pueden ser botones, etiquetas, campos de texto, etc. Aquí tienes algunos widgets comunes:

Etiqueta (Label): Muestra texto o imágenes en la ventana.

```
9 etiqueta = tk.Label(ventana, text="Hola, Tkinter")
10 etiqueta.pack() # Agrega el widget a la ventana
```

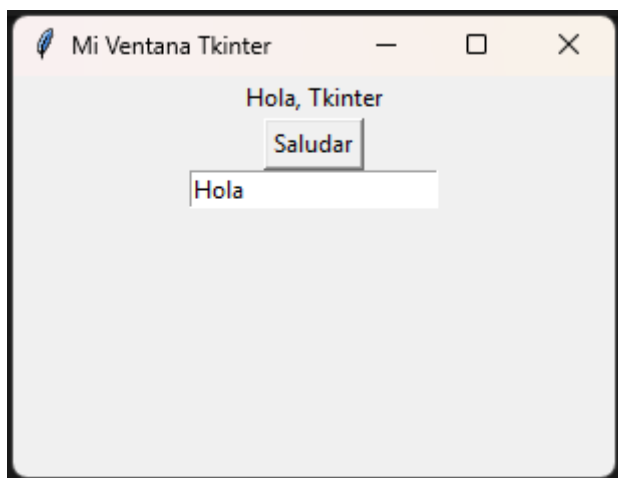
Botón (Button): Permite al usuario hacer clic para ejecutar una acción.

```
boton = tk.Button(ventana, text="Saludar", command=saludar)
boton.pack() # Agregamos el boton a la ventana
```

Entrada de texto (Entry): Permite al usuario escribir texto en un campo de entrada.

```
entrada = tk.Entry(ventana)
entrada.pack() # Agregamos la entrada de texto a la ventana
```

Resultado final:



Colocación de Widgets (pack, grid, place): Para que los widgets aparezcan en la ventana, debes colocarlos usando algún método de geometría. Los más comunes son:

- **pack():** Coloca el widget automáticamente en el siguiente espacio disponible.
etiqueta.pack()
- **grid():** Coloca los widgets en una cuadrícula (grid). Se define usando filas y columnas.
etiqueta.grid(row=0, column=0)
- **place():** Coloca el widget en una posición exacta dentro de la ventana.
etiqueta.place(x=50, y=50)

Eventos y Comandos

Tkinter permite asignar eventos o funciones que se ejecutan cuando el usuario interactúa con los widgets. Un ejemplo común es usar el parámetro **command** en un botón para vincularlo a una función que se ejecutará cuando el usuario haga clic.

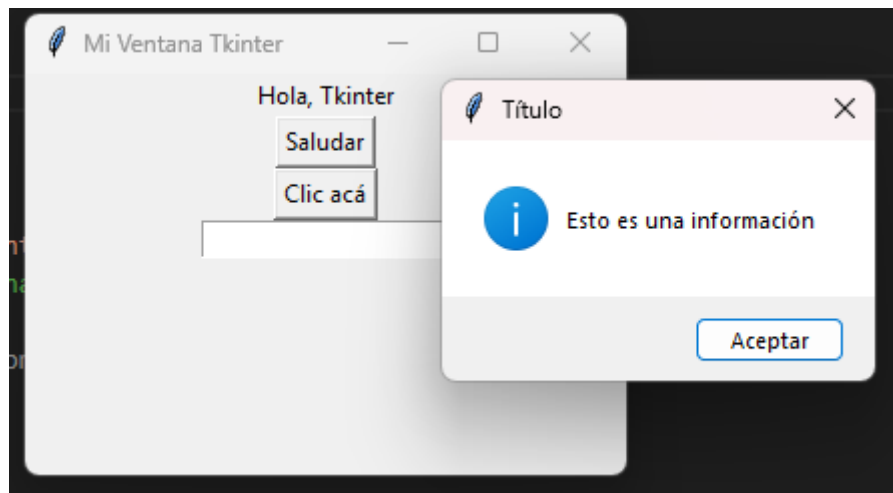
Ventana de diálogo

Tkinter tiene un módulo llamado messagebox que permite mostrar ventanas emergentes con mensajes al usuario, como notificaciones de éxito o error.

```
def mostrar_mensaje():  
    messagebox.showinfo("Título", "Esto es una información")  
    messagebox.showerror("Error", "Hubo un problema")  
  
boton = tk.Button(ventana, text="Clic acá", command=mostrar_mensaje)  
boton.pack()
```

Código que atiende el evento de click en el botón y ejecuta la función que muestra los messagebox.

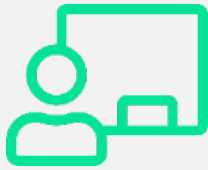
Resultado:





Hemos llegado así al final de esta clase en la que vimos:

- Aprendimos a modelar funcionalmente un sistema a través de casos de uso.
- Vimos cómo utilizar relaciones include y extend.
- Comprendimos la diferencia entre flujos alternativos y extensiones.
- Introdujimos la noción de subsistemas como unidades dentro de un sistema mayor.
- Comparamos dos grandes enfoques: análisis estructurado y orientado a objetos.
- Reforzamos la importancia de la trazabilidad y el balanceo para garantizar coherencia en los modelos.
- Aprendimos cómo se define el punto de entrada de una aplicación con `main()` y por qué conviene usarlo.
- Trabajamos con diccionarios, una estructura muy flexible y fundamental en Python.
- Vimos cómo reutilizar código importando clases desde otros archivos.
- Exploramos los conceptos básicos de Tkinter, abriendo la puerta a crear interfaces gráficas en Python.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.
¡Hasta la próxima clase!

Bibliografía

Chen, P. P. (1976). The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36.

<https://doi.org/10.1145/320434.320440>

Pressman, R. S., & Maxim, B. R. (2015). *Ingeniería de software: Un enfoque práctico* (8.ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Vega, M. (s. f.). *Casos de uso*. Universidad de Granada.

<https://lsi2.ugr.es/~mvega/docis/casos%20de%20uso.pdf>

Python Software Foundation. (2024). Tkinter — Python interface to Tcl/Tk. Python 3.13 documentation. Recuperado de

<https://docs.python.org/es/3.13/library/tkinter.html>

StarUML. (s. f.). Herramienta de modelado UML. Recuperado de

<https://staruml.io/download/>

Lucidchart. (s. f.). Cómo empezar a usar Lucidchart. Recuperado de

<https://www.lucidchart.com/blog/es/como-empezar-a-usar-lucidchart>

Lucidchart. (s. f.). Herramienta online para crear diagramas UML.

<https://lucid.app/es/pricing/lucidchart>

diagrams.net. (s. f.). Aplicación gratuita para crear diagramas UML.

<https://app.diagrams.net>

MGPanel. (s. f.). ¿Qué es la programación orientada a objetos?.

<https://blog.mgpanel.org/post/-que-es-la-programacion-orientada-a-objetos->